AI Product Intelligence System

Gen AI Bootcamp — Day 2 Homework Submission

Advanced Features: Complementary Recommendations, Duplicate Catalog Cleanup & Text-Based Reverse Search

Author	Manish R
Dataset	Fashion Product Images (Small) — Kaggle
Model Used	CLIP (openai/clip-vit-base-patch32)
Kaggle Notebook	https://www.kaggle.com/code/manishr00799/notebook4d97fc13a9/

Overview

The base system understands products from images, generates product metadata, and finds visually similar products using CLIP embeddings and vector search. This submission extends that system with all three advanced features from the Day 2 challenge: a complementary-product recommendation engine, an automatic duplicate/near-duplicate catalog cleaner, and a text-to-image reverse search interface.

Approach in Brief

- Loaded 200 sampled products from styles.csv and matched each to its image on disk, skipping any missing files.
- Generated a 512-dimensional, L2-normalized CLIP image embedding for every product using openai/clip-vit-base-patch32.
- Task 1 uses a category-pairing map (e.g. Tshirts → Jeans, Casual Shoes, Watches) combined with cosine similarity within the paired categories to recommend items that are commonly worn together, not just visually similar.
- Task 2 builds a cosine-similarity matrix across all embeddings and greedily collapses any product with similarity ≥ 0.92 to an already-kept item, leaving one representative per near-duplicate cluster.
- Task 3 encodes a free-text query with CLIP's text tower into the same embedding space as the images, then ranks all products by cosine similarity to that text vector.

Task 1 — Smart Product Recommendation Engine

Problem Statement

The base system only finds visually similar products. E-commerce platforms also recommend products that are commonly purchased together (e.g. Running Shoe → Sports Socks, Fitness Watch, Water Bottle). The task was to build a recommender for these complementary products.

How the Recommendations Are Generated

A COMPLEMENTARY_MAPPING dictionary defines which article types pair with which others (for example, Eyewear pairs with Watches, Belts, Perfume and Caps; Tshirts pair with Jeans, Casual Shoes, Watches and Jackets). For a given input product, the system looks up its articleType, filters the catalog down to only the items belonging to the paired categories, and then ranks those candidates by cosine similarity between their CLIP embeddings and the input product's embedding. This ensures recommendations are both categorically complementary (a genuinely different, pairable item type) and visually/stylistically coherent with the input (similar colour, pattern or aesthetic within that category). If no mapping exists for a category, the system falls back to recommending from the full catalog.

Result

Input product: a women's classic sunglasses (Eyewear). The three recommended items are all from the mapped complementary categories and are visually close in style to the input, confirming the logic works as intended.

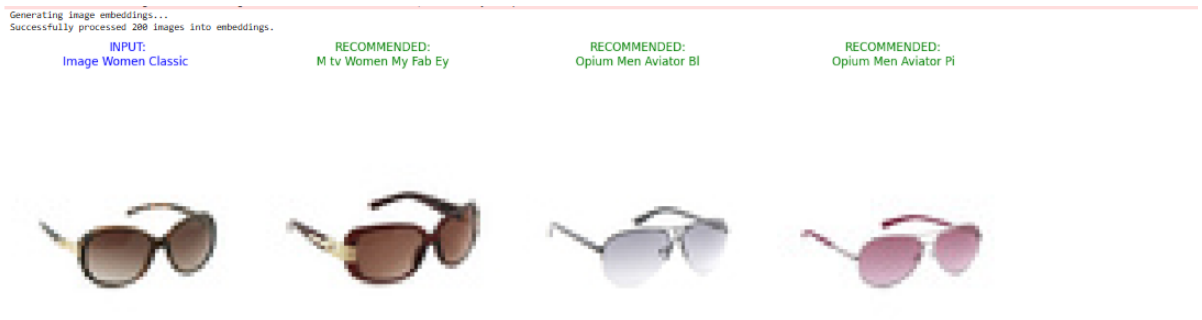

Figure 1: Input sunglasses (blue) with three complementary recommendations (green).

Task 2 — Unique Product Catalog Creation

Problem Statement

Large marketplaces often contain duplicate or near-duplicate product listings uploaded by different sellers (e.g. three near-identical blue shirts). The task was to build a system that automatically produces a clean catalog containing only unique products.

How Duplicates Are Identified and Removed

A full pairwise cosine-similarity matrix is computed across all 200 CLIP image embeddings. The catalog is then walked in order: each unvisited product is kept as a representative, and every other product whose similarity to it is 0.92 or higher is marked visited and excluded going forward. This effectively clusters near-duplicate listings (same or nearly identical product photographed/listed multiple times) and keeps a single representative per cluster, producing a deduplicated catalog.

Result

Out of the 200 sampled products, the deduplication step reduced the catalog to 151 unique products — removing 49 near-duplicate listings (similarity ≥ 0.92).

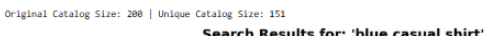

Figure 2: Catalog size before and after deduplication.

Task 3 — Reverse Product Search

Problem Statement

Instead of searching with an image, users should be able to search the catalog using a free-text query, e.g. “blue casual shirt”, and get back the most relevant matching products.

How Text-to-Image Search Works

CLIP's text encoder projects the input query into the same embedding space used for the product images. The query embedding is L2-normalized and compared against every pre-computed, normalized image embedding using cosine similarity. The top-k highest scoring products are returned and displayed along with their similarity scores, giving a ranked, text-driven search experience without needing any labelled search index.

Result

Query: “blue casual shirt”. The top 3 results returned were all blue/light-toned men's casual shirts, with similarity scores of 0.2780, 0.2725 and 0.2723 respectively — confirming the text query correctly retrieved semantically and visually relevant products.

Figure 3: Top 3 results for the text query “blue casual shirt”, with cosine similarity scores.

Appendix — Source Code

Full source code of the notebook implementing the base pipeline (data loading, CLIP embedding generation) and all three advanced-feature tasks described above.

```
import os
import torch
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image, ImageFile
from sklearn.metrics.pairwise import cosine_similarity
from transformers import CLIPProcessor, CLIPModel

# Tell PIL to be lenient with broken/truncated image files
ImageFile.LOAD_TRUNCATED_IMAGES = True

# 1. SETUP & INITIALIZATION
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {device}")

# Absolute paths based on your exact environment path strings
CSV_PATH = '/kaggle/input/datasets/paramaggarwal/fashion-product-images-small/styles.csv'
IMAGES_DIR = '/kaggle/input/datasets/paramaggarwal/fashion-product-images-small/images'

# Load dataset and handle bad formatting lines gracefully
df = pd.read_csv(CSV_PATH, on_bad_lines='skip')

# Ensure the 'id' column is treated cleanly as integer strings matching image names
df = df.dropna(subset=['id'])
df['id'] = df['id'].astype(int)
df['image_path'] = df['id'].apply(lambda x: os.path.join(IMAGES_DIR, f"{x}.jpg"))

# Filter down to entries where the image file actually exists on disk
df = df[df['image_path'].apply(os.path.exists)].reset_index(drop=True)
print(f"Total valid images found matching CSV entries: {len(df)}")

if len(df) == 0:
    raise ValueError("The dataframe is empty after filtering for existing images. Double check
the folder paths.")

# Sample a smaller subset (e.g., 200) to keep it fast on CPU
sample_size = min(200, len(df))
df = df.sample(n=sample_size, random_state=42).reset_index(drop=True)

# Load CLIP Model
model_name = "openai/clip-vit-base-patch32"
model = CLIPModel.from_pretrained(model_name).to(device)
processor = CLIPProcessor.from_pretrained(model_name)

# 2. GENERATE EMBEDDINGS
image_embeddings = []
valid_indices = []

print("Generating image embeddings...")
for idx, row in df.iterrows():
    try:
        # Open and force conversion to standard RGB (handles grayscale/RGBA variants)
        img = Image.open(row['image_path']).convert('RGB')

        # Process through CLIP
```

```
        inputs = processor(images=img, return_tensors="pt").to(device)

        with torch.no_grad():
            img_emb = model.get_image_features(**inputs)

        # Handle cases where the returned object is a wrapper instead of a raw tensor
        if hasattr(img_emb, 'pooler_output'):
            img_emb = img_emb.pooler_output
        elif hasattr(img_emb, 'image_embeds'):
            img_emb = img_emb.image_embeds

        # Normalize the vector
        img_emb = img_emb / img_emb.norm(p=2, dim=-1, keepdim=True)

        image_embeddings.append(img_emb.cpu().numpy().flatten())
        valid_indices.append(idx)

    except Exception as e:
        print(f"Skipping image {row['id']}.jpg due to error: {e}")
        continue

# Keep only the rows that successfully passed through the embedding generator
df = df.iloc[valid_indices].reset_index(drop=True)

if len(df) == 0:
    raise ValueError("Still failed to process any images. Check the error prints above for clue
s.")

image_embeddings = np.array(image_embeddings)
print(f"Successfully processed {len(df)} images into embeddings.")

# ==========================================
# TASK 1: SMART PRODUCT RECOMMENDATION ENGINE
# ==========================================
# Updated Complementary Mapping to explicitly cover Eyewear and other types
COMPLEMENTARY_MAPPING = {
    'Eyewear': ['Watches', 'Belts', 'Perfume', 'Caps'],
    'Casual Shoes': ['Socks', 'Watches', 'Backpacks'],
    'Sports Shoes': ['Socks', 'Watches', 'Sports Wear'],
    'Tshirts': ['Jeans', 'Casual Shoes', 'Watches', 'Jackets'],
    'Shirts': ['Trousers', 'Formal Shoes', 'Watches', 'Belts'],
    'Jeans': ['Tshirts', 'Casual Shoes', 'Belts']
}

def recommend_complementary_products(product_id, top_n=3):
    product_row = df[df['id'] == product_id].iloc[0]
    category = product_row['articleType']

    target_categories = COMPLEMENTARY_MAPPING.get(category, [])
    filtered_df = df[(df['articleType'].isin(target_categories)) & (df['id'] != product_id)]

    if filtered_df.empty:
        filtered_df = df[df['id'] != product_id]

    filtered_indices = filtered_df.index.tolist()
    input_idx = df[df['id'] == product_id].index[0]
```

```
    input_emb = image_embeddings[input_idx].reshape(1, -1)
    target_embs = image_embeddings[filtered_indices]

    sims = cosine_similarity(input_emb, target_embs).flatten()
    top_local_indices = np.argsort(sims)[::-1][:top_n]

    return filtered_df.iloc[top_local_indices]

# Run Task 1 Execution
if len(df) > 0:
    sample_id = df.iloc[0]['id']
    rec_results = recommend_complementary_products(product_id=sample_id, top_n=3)

    fig, axes = plt.subplots(1, min(len(rec_results) + 1, 4), figsize=(15, 5))
    if len(rec_results) == 0:
        axes = [axes]

    axes[0].imshow(Image.open(df[df['id']==sample_id]['image_path'].values[0]).convert('RGB'))
    axes[0].set_title(f"INPUT:\n{df[df['id']==sample_id]['productDisplayName'].values[0][:20]}"
, color='blue')
    axes[0].axis('off')

    for i, (_, row) in enumerate(rec_results.iterrows()):
        if i + 1 < len(axes):
            ax = axes[i+1]
            ax.imshow(Image.open(row['image_path']).convert('RGB'))
            ax.set_title(f"RECOMMENDED:\n{row['productDisplayName'][:20]}", color='green')
            ax.axis('off')
    plt.tight_layout()
    plt.show()

# ========================================
# TASK 2: UNIQUE PRODUCT CATALOG CREATION
# ========================================
def generate_unique_catalog(embeddings, dataframe, similarity_threshold=0.92):
    if len(dataframe) == 0:
        return dataframe
    sim_matrix = cosine_similarity(embeddings)
    visited = set()
    unique_indices = []

    for i in range(len(dataframe)):
        if i in visited:
            continue
        similar_idx = np.where(sim_matrix[i] >= similarity_threshold)[0]
        for idx in similar_idx:
            visited.add(idx)
        unique_indices.append(i)

    return dataframe.iloc[unique_indices].copy()

clean_catalog = generate_unique_catalog(image_embeddings, df)
print(f"Original Catalog Size: {len(df)} | Unique Catalog Size: {len(clean_catalog)}")

# ========================================
```

```
# TASK 3: REVERSE PRODUCT SEARCH
# ========================================
def text_reverse_search(query_text, top_k=3):
    if len(df) == 0:
        return pd.DataFrame()
    inputs = processor(text=[query_text], return_tensors="pt", padding=True).to(device)
    with torch.no_grad():
        text_emb = model.get_text_features(**inputs)

    if hasattr(text_emb, 'pooler_output'):
        text_emb = text_emb.pooler_output
    elif hasattr(text_emb, 'text_embeds'):
        text_emb = text_emb.text_embeds

    text_emb = text_emb / text_emb.norm(p=2, dim=-1, keepdim=True)
    text_emb = text_emb.cpu().numpy()

    similarities = cosine_similarity(text_emb, image_embeddings).flatten()
    top_indices = np.argsort(similarities)[::-1][:top_k]

    results = df.iloc[top_indices].copy()
    results['similarity_score'] = similarities[top_indices]
    return results

# Run Task 3 Execution
if len(df) > 0:
    search_query = "blue casual shirt"
    search_results = text_reverse_search(search_query, top_k=3)

    if not search_results.empty:
        fig, axes = plt.subplots(1, len(search_results), figsize=(12, 4))
        if len(search_results) == 1:
            axes = [axes]
        fig.suptitle(f"Search Results for: '{search_query}'", fontsize=14, fontweight='bold')

        for i, (_, row) in enumerate(search_results.iterrows()):
            ax = axes[i]
            ax.imshow(Image.open(row['image_path']).convert('RGB'))
            ax.set_title(f"{row['productDisplayName'][:20]}\nScore: {row['similarity_score']:.4
f}")
            ax.axis('off')
        plt.tight_layout()
        plt.show()
```